ИТМО

Управление потоком выполнения, коллекции и структуры данных

Терещенко Владислав Витальевич, ассистент ФПИн

TG: VI_Tershch
vvtereshchenko@itmo.ru
Отчеты отправлять на: vlad-tershch@yandex.ru

Pattern Matching

Зачастую приходится проверять не только тип данных, но и количество, что приводит к появлению огромного числа веток if/else с вызовом функций isinstance, len и обращению к элементам по индексу, ключу или атрибуту.

Решение -> match/case

Pattern Matching

```
def load():
    print("Загружаем")
def save():
    print("Сохраняем")
def default():
    print("Неизвестно как обработать")

def main(value):
    if isinstance(value, str) and value == "load":
        load()
    elif isinstance(value, str) and value == "save":
        save()
    else:
        default()

main("load")
>>> Загружаем
main("save")
>>> Сохраняем
main("hello")
>>> Неизвестно как обработать
```

```
def main(value):
    match value:
        case "load":
            load()
        case "save":
            save()
        case _:
            default()
main("load")
>>> Загружаем
main("save")
>>> Сохраняем
main(5645)
>>> Неизвестно как обработать
```

Pattern Matching

```
def main(data_string):
   values = data_string.split("~")
   match values:
       case name, "1"|"2" as access, request:
           print(f"Пользователь {name} получил доступ к функции {request} с правами {
       case _:
           print("Неудача")
main("Daniil~2~load")
>>> Пользователь Daniil получил доступ к функции load с правами 2
main("Kris~0~save")
>>> Неудача
```

```
def main(data_dict):
   match data_dict:
       case {"name": str(name), "access": 1|2 as access, "request": request}:
           print(f"Пользователь {name} получил доступ к функции {request} с правами {
       case _:
           print("Неудача")
main({"name": "Daniil", "access": 1, "request": "save"})
>>> Пользователь Daniil получил доступ к функции save с правами 1
main({"name": ["Daniil"], "access": 1, "request": "save"})
>>> Неудача
main({"name": "Kris", "access": 0, "request": "load"})
>>> Неудача
```

Немного про циклы

```
print(range(5)) # ответ:
range(0, 5)

a = range(5, 10)
print(a)
# range(5, 10)

b = list(range(1, 10, 2))
print(b) # [1, 3, 5, 7, 9]
```

```
for number in range(5):
print(number)

==

for number in [0, 1, 2, 3, 4]:
print(number)
```

Немного про циклы

```
a_dict = {"one":1, "two":2,
"three":3} for key in a_dict:
        print(key)
```

```
i = 0
while i < 10:
        print(i) i = i + 1
```

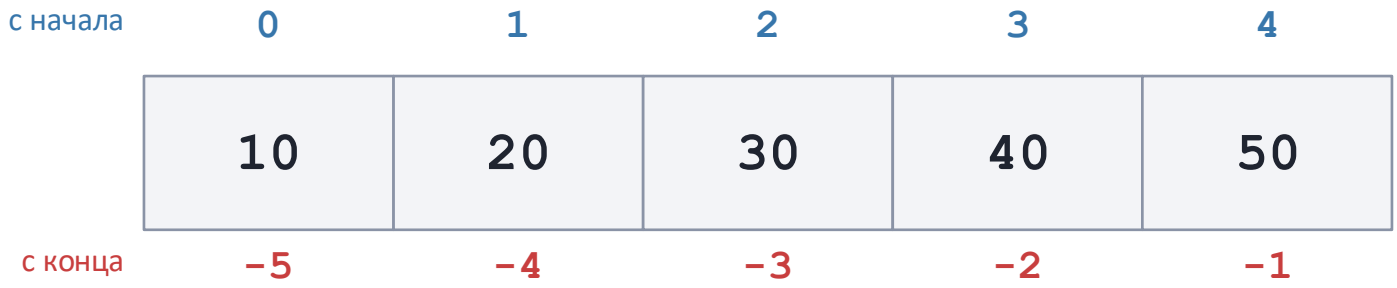
```
while i < 10:
        print(i)
        if i == 5:
        break
        i += 1
```

Немного про циклы

Оператор	Что делает	Куда переходит управление
`break`	Полностью останавливает цикл	На код сразу после цикла (блок `else` пропускается)
`continue`	Пропускает остаток текущего шага	На начало следующей итерации этого же цикла

Важно: И break, и continue можно использовать **только** внутри циклов for и while. Если написать их вне цикла, Python выдаст ошибку SyntaxError

Индекс: позиция с начала или с конца




```
nums = [10, 20, 30, 40, 50]
nums[0]     # 10
nums[-1]    # 50
nums[5]     # IndexError
```

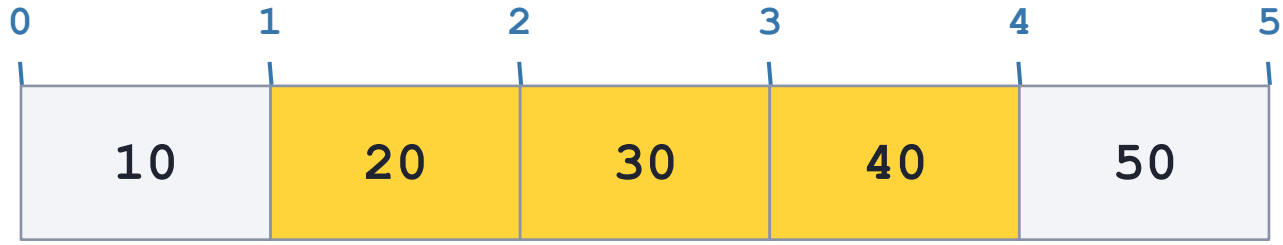
Только последовательности

list, tuple, str, range, bytes.

set[0] → TypeError: порядка нет.

dict[0] ищет ключ 0, а не позицию.

Срез [start:stop:step]



nums[1:4] → [20, 30, 40]

```
nums[::2]     # [10, 30, 50]
nums[::-1]    # [50, 40, 30, 20, 10]
nums[1:100]   # [20, 30, 40, 50]
nums[3:1]     # []
```

start включается,

stop — нет

`len(nums[a:b]) == b - a`

Шпаргалка по срезам

`[:n]`	первые n
`[-n:]`	последние n
`[::2]`	каждый второй
`[::-1]`	в обратном порядке
`[:]`	копия целиком

Срез никогда не даёт IndexError

- Создаёт новый объект того же типа: срез строки — строка.
- Копия поверхностная: копируются ссылки, а не элементы.
- `s[1:4] == s[slice(1, 4)]`

Присваивание срезу

```
a = [1, 2, 3, 4, 5]
a[1:3] = ['x', 'y', 'z']
# [1, 'x', 'y', 'z', 4, 5]
a[:2] = []
# ['y', 'z', 4, 5]
a[::2] = [0, 0]
# [0, 'z', 0, 5]
del a[::2]
# ['z', 5]
```

list

можно менять срезом

tuple, str

TypeError: неизменяемые

шаг ≠ 1

длины слева и справа обязаны совпасть

Методы: на месте или новый объект

Тип	Меняют на месте → None	Возвращают значение
`list`	append, extend, insert, remove, sort, reverse, clear	pop, index, count, copy
`tuple`	— (неизменяемый)	index, count
`set`	add, remove, discard, update, pop	union, intersection, difference
`dict`	update, setdefault, pop, popitem	get, keys, values, items, copy

`nums = nums.sort()` → nums теперь None. Нужен новый список? `sorted(nums)`

Безопасные методы и операторы

Падает

`d[k]`
KeyError

`s.remove(x)`
KeyError

`l.index(x)`
ValueError

Не падает

`d.get(k, 0)`

`s.discard(x)`

`x in l`

Множества

`a | b`	объединение
`a & b`	пересечение
`a - b`	разность
`a ^ b`	симм. разность

`d1 | d2` — словари, Python 3.9+

Четыре формы вложенности

Матрица · таблица чисел

```
m = [[1, 2], [3, 4]]
m[1][0]  # 3
```

Список словарей · JSON

```
users = [{'name': 'Анна'}]
users[0]['name']
```

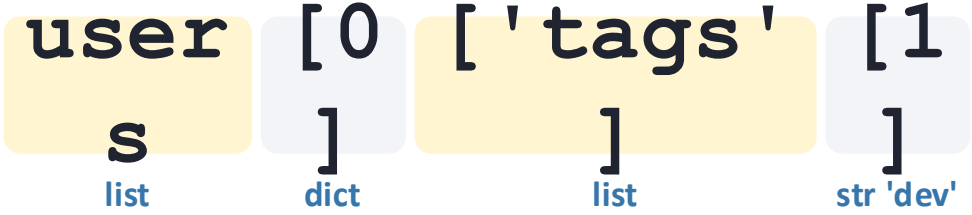
Словарь списков · группировка

```
by_dept = {'dev': ['Анна']}
by_dept['dev'][0]
```

Словарь словарей · конфиг

```
cfg = {'db': {'port': 5432}}
cfg['db']['port']
```

Читаем цепочку обращений



тип после каждого шага

```
users = [
    {'name': 'Анна', 'tags': ['admin', 'dev']},
    {'name': 'Борис', 'tags': ['dev']},
]
```

`config.get('db', {}).get('port', 5432)` — безопасно на 2–3 уровня

Строим и обходим

Построение: группировка

```
from collections import defaultdict

groups = defaultdict(list)
for u in users:
    for tag in u['tags']:
        groups[tag].append(u['name'])

# {'admin': ['Анна'],
#  'dev': ['Анна', 'Борис']}
```

Обход: генераторы

```
m = [[1, 2], [3, 4]]

flat = [x for r in m for x in r]
# [1, 2, 3, 4]

t = [list(c) for c in zip(*m)]
# [[1, 3], [2, 4]]
```

Две ловушки вложенности

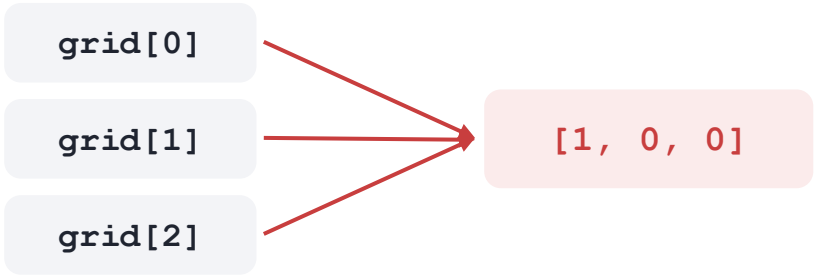
Кортеж неизменяем только «снаружи»

```
t = (1, [2, 3])
t[1].append(4)
# (1, [2, 3, 4]) — работает
t[1] = [9]
# TypeError
```

Значит, hash(t) упадёт — такой кортеж не положить в set.

Три ссылки на один список

```
grid = [[0] * 3] * 3
grid[0][0] = 1
# [[1,0,0],[1,0,0],[1,0,0]]
```



Решение: `[[0] * 3 for _ in range(3)]`

Задача → структура

Задача	Структура	Почему
Упорядоченный изменяемый набор	`list`	порядок, append, индекс
Фиксированная запись	`tuple`	неизменяем, может быть ключом
Уникальные значения, «есть ли x»	`set`	x in s — в среднем O(1)
Неизменяемое множество	`frozenset`	хешируемое
Поиск значения по ключу	`dict`	d[key] — в среднем O(1)
Подсчёт количества	`Counter`	most_common()
Очередь FIFO	`deque`	popleft() — O(1)
Стек LIFO	`list`	append / pop() — O(1)

in: список против множества

```
data = list(range(100_000))
s = set(data)

99_999 in data   # перебор, O(n)
99_999 in s      # хеш, O(1)
```

Чем больше данных, тем больше разрыв: список растёт линейно, множество — нет.

≈495 мкс

list

≈55 нс

set

≈9 000×

быстрее на одной проверке

Python 3.13, 100 000 элементов, худший случай для списка

Цена быстрого поиска и анти-паттерны

Чем платим

- Больше памяти, чем у списка
- Элементы должны быть хешируемыми
- set не хранит порядок и дубли
- set(list) стоит O(n): нужен, если проверок много

список пар + цикл поиска

→ `dict`

lst.pop(0) для очереди

→ `deque.popleft()`

list(set(x)), порядок важен

→ `list(dict.fromkeys(x))`

dict с ключами 0, 1, 2…

→ `list`

Определение и контракт

1. Есть __hash__, hash(obj) возвращает int
2. Хеш не меняется, пока объект жив
3. Есть __eq__, и выполняется контракт

```
если a == b

то hash(a) == hash(b)
```

Обратное неверно. Равные хеши у разных объектов — это коллизия.

Хешируемые и нехешируемые типы

Хешируемые

```
int  float  bool  None
str  bytes
frozenset
tuple*
функции, классы, объекты
```

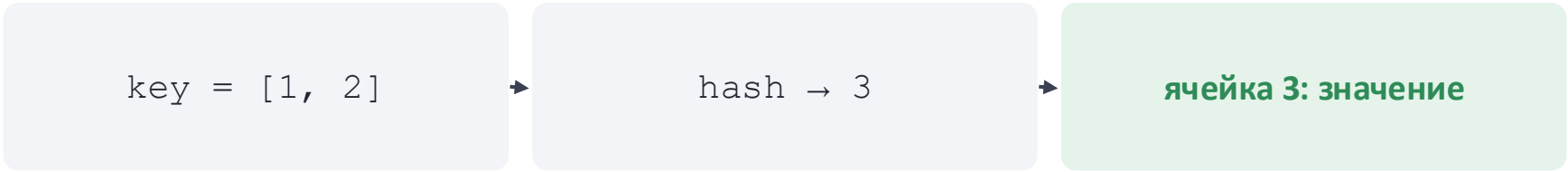
Нехешируемые

```
list
dict
set
bytearray
```

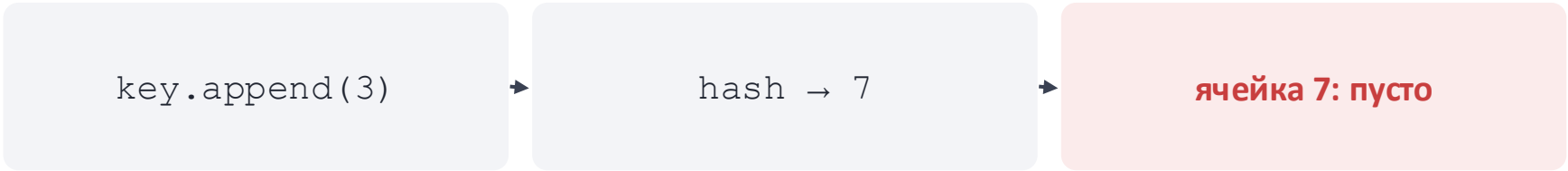
tuple* — только если все элементы хешируемы: **`hash((1, [2]))`** → **TypeError**

Почему список не может быть ключом

Положили ключ



Изменили ключ и ищем снова



Ключ остался в словаре, но найти его нельзя. Поэтому Python запрещает это заранее: TypeError.

Свои классы: __eq__ отключает __hash__

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return (self.x, self.y) == \
               (other.x, other.y)

{Point(1, 2)}
# TypeError: unhashable type
```

Способ 1: свой __hash__

```
def __hash__(self):
    return hash((self.x, self.y))
```

Способ 2: dataclass

```
@dataclass(frozen=True)
class Point:
    x: int
    y: int
```

Хеш — из тех же полей, что и равенство

Что делает хеш-функция



Детерминированность

тот же объект — тот же хеш

Скорость

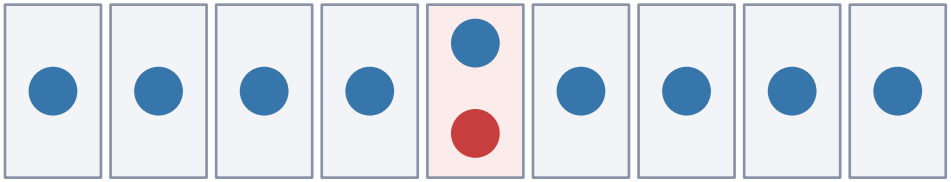
не зависит от размера коллекции

Равномерность

похожие входы — разные хеши

Коллизии неизбежны

10 объектов, 9 ячеек



Входов бесконечно много, а значений хеша — конечное число. Где-то совпадение обязательно.

```
hash(-1)    # -2
hash(-2)    # -2
```

В CPython -1 зарезервирован как код ошибки

Поэтому словарь проверяет ключ в два шага: **`hash`** — найти ячейку, **`==`** — убедиться, что это тот самый ключ.

Как Python хеширует встроенные типы

Числа согласованы

```
hash(42)          # 42
{1, 1.0, True}    # {1}
d = {1: 'a'}
d[True] = 'b'     # {1: 'b'}
```

Строки меняют хеш при каждом запуске

```
$ python -c "print(hash('abc'))"
-3889881146889034134
$ python -c "print(hash('abc'))"
1945849499562701049
```

Не сохраняйте hash() в файл или БД

Строки хешируются SipHash со случайным зерном (Python 3.3+, PYTHONHASHSEED). Это защита от hash flooding: нельзя заранее подобрать тысячи ключей с одним хешем.

hash() и hashlib — разные инструменты

	`hash()`	`hashlib.sha256`
Назначение	dict, set	целостность, подписи
Результат	int, 64 бита	256 бит, hexdigest()
Стабилен между запусками	нет (для str)	да
Устойчив к подбору коллизий	не цель	да

```
hashlib.sha256('текст'.encode()).hexdigest()   # одинаково всегда
```

Пароли: только scrypt, bcrypt или Argon2 — с солью и медленно

Размер шрифта

Спасибо за внимание!

TG: Vl_Tershch
vvtereshchenko@itmo.ru
Отчеты отправлять на: vlad-tershch@yandex.ru

ITMO more than a UNIVERSITY